

M1.1: MySQL vs PostgreSQL (关系型数据抉择)

Theory: MySQL uses index-organized tables (B+ Tree) with secondary index lookups, relying on Undo Log chains for MVCC. PostgreSQL uses heap tables with version meta-data (xmin/xmax) in the shared buffer (page-inline writes).

Details: MySQL InnoDB blocks phantoms via Gap Locking in Repeatable Read isolation. PostgreSQL employs SSI (Serializable Snapshot Isolation) based on SIREAD lock tracking for dependency cycles, cleaning dead tuples via VACUUM.

Trade-off: MySQL excels at frequent index lookups and simple writes; PostgreSQL supports advanced SQL and custom types but suffers from WAL overhead and Table Bloat.

M1.3: Redis vs Memcached (分布式缓存抉择)

Theory: Redis is a single-threaded event-loop cache supporting data structures, persistence, and replication. Memcached is a multi-threaded Slab Allocation key-value memory store without native persistence.

Details: Redis relies on epoll multiplexing and RDB/AOF pipelines for durability. Memcached uses LRU linked lists and Slab isolation to mitigate memory fragmentation, limiting item value sizes to 1MB.

Trade-off: Redis is feature-rich but single-thread blocks on large keys. Memcached is lightweight and ultra-fast for simple lookups but sharding must be handled by client hash rings.

M1.5: RocksDB vs LevelDB (单机嵌入式存储抉择)

Theory: RocksDB is a high-performance LSM-Tree KV store optimized for multi-core CPUs and fast SSDs. LevelDB is a lightweight single-threaded LSM engine.

Details: RocksDB introduces Column Families, parallel MemTable writing, and concurrent Compaction threads. LevelDB relies on a single-threaded merge loop.

Trade-off: RocksDB offers extreme write throughput but configuration is complex. LevelDB is simple and highly stable for small-footprint runtimes.

M2.1: Kafka vs Apache Pulsar (高性能事件流抉择)

Theory: Kafka couples storage and brokers using page-cache and zero-copy append logs. Pulsar decouples compute (stateless Brokers) from storage (distributed BookKeeper Bookies).

Details: Kafka partitions are bound to static Segment files on specific Brokers. Pulsar partitions are divided into Ledgers distributed across Bookie pools.

Trade-off: Kafka is simple and extremely fast but scaling requires partition rebalancing. Pulsar allows instant scaling and supports millions of partitions but the system path is deep.

M1.2: MongoDB vs Cassandra (NoSQL 抉择)

Theory: MongoDB is a strongly-consistent document DB using B-Trees. Cassandra is a high-availability wide-column store using Dynamo-style peer-to-peer hashing rings, SSTables, and Gossip protocols for eventual consistency.

Details: MongoDB uses WiredTiger engine locking and Replica Set elections. Cassandra balances consistency via client read/write parameters ($Quorum = R + W > N$), writing to CommitLogs and MemTables.

Trade-off: MongoDB fits dynamic schemas requiring ACID transactions. Cassandra provides infinite write scaling and zero single-points-of-failure but lacks complex transactions and suffers from Read Repair lag.

M1.4: InfluxDB vs TimescaleDB (时序数据库抉择)

Theory: InfluxDB uses a purpose-built column-oriented TSM/TSI engine for metrics. TimescaleDB extends PostgreSQL with Hypertables for relational time-series support.

Details: InfluxDB serializes metrics into TSM files with TSI indexing for tags. TimescaleDB breaks massive tables into hypertable Chunks sorted by time, inheriting PostgreSQL's B-Tree indexing.

Trade-off: InfluxDB yields high compression and write scaling but memory explodes on high-cardinality tags. TimescaleDB supports full SQL JOINS but storage overhead is higher.

M1.6: CockroachDB vs TiDB (分布式 NewSQL 抉择)

Theory: CockroachDB uses a peer-to-peer Range architecture backed by Pebble KV engines. TiDB separates computation (stateless SQL layer) from storage (Multi-Raft TiKV and column-oriented TiFlash).

Details: CockroachDB coordinates transactions via HLCs and 2PC with write intent locks. TiDB uses a centralized Placement Driver (PD) for TSO clocks and Percolator transactional pipelines.

Trade-off: CockroachDB provides excellent multi-datacenter active-active survivability. TiDB has a mature ecosystem for HTAP but deployment is complex with many moving parts.

M2.2: RocketMQ vs RabbitMQ (企业级队列抉择)

Theory: RocketMQ utilizes mixed CommitLog writes with consumer queue indexes and consumer polling. RabbitMQ uses AMQP and Erlang Actor pools to push events to consumers.

Details: RocketMQ stores all topics in a single CommitLog, indexing them for consumer poll loops. RabbitMQ routes messages via Exchanges to Memory Queues, pushing to channels.

Trade-off: RocketMQ handles massive message backlogs and transaction tokens easily. RabbitMQ offers lower latency for small payloads but fails to handle large storage backlogs.

M2.4: Event Sourcing vs Event-Driven (事件流转抉择)

Theory: Event Sourcing logs every state change as an immutable event stream, calculating current state via replays. Event-Driven Architecture (EDA) relies on transient events to decouple decoupled systems.

Details: Event Sourcing requires an Event Store and snapshots for fast reconstruction. EDA routes events via message brokers and drops them upon consumer ACK.

Trade-off: Event Sourcing provides complete audit trails but schema evolution is difficult. EDA is flexible but eventual consistency makes debugging harder.

M2.3: Redis Streams vs MQTT (轻量/物联网队列抉择)

Theory: Redis Streams uses a memory-based append-only log structure. MQTT is a lightweight publish/subscribe transport protocol designed for constrained IoT sensors.

Details: Redis Streams indexes entries via Radix Trees, tracking consumer groups and ACKs. MQTT coordinates connections over TCP, routing events using QoS levels 0/1/2.

Trade-off: Redis Streams is perfect for simple in-memory RPC queues. MQTT scales to millions of IoT devices but requires external brokers to manage cluster consensus.

M2.5: Push vs Pull Consumer (消费者驱动模式)

Theory: The push model delivers messages from broker to consumer sockets. The pull model forces consumers to request message batches based on capacity.

Details: Push models stream packets over long connections, risking buffer overflow on slow clients. Pull models (e.g. Kafka long polling) balance consumer latency and resource overhead.

Trade-off: Push minimizes ingestion latency but requires broker flow control. Pull protects consumer runtimes but introduces poll-loop overhead.

M2.6: Partition-based vs Hash Ring (分片路由抉择)

Theory: Partition sharding hashes keys to static partitions. Hash ring sharding maps keys and nodes to a circular hash space.

Details: Adding partitions to partition-sharded clusters requires complete data redistribution. Hash rings allow key drift to adjacent nodes when topology changes.

Trade-off: Partition sharding guarantees strict order per segment. Hash rings offer elastic scaling but require virtual nodes to mitigate hot spots.

M3.1: RESTful HTTP/1.1 vs gRPC HTTP/2 (传输协议抉择)

Theory: RESTful defaults to HTTP/1.1 JSON text payloads over Keep-Alive connections. gRPC is contract-based, using binary Protobuf payloads over HTTP/2 multiplexed streams.

Details: RESTful relies on URI paths and verbs. gRPC generates typed client/server stubs via 'proto' definitions, avoiding head-of-line blocking.

Trade-off: RESTful is human-readable and browser-friendly. gRPC is high-performance for internal RPC networks but requires transcoding for standard Web clients.

M3.2: GraphQL vs RESTful API (网关 API 抉择)

Theory: GraphQL exposes a single endpoint for declarative query schemas. RESTful maps resources to distinct HTTP endpoints.

Details: GraphQL resolve functions process query DSLs to fetch related structures in one request. RESTful requires clients to invoke multiple endpoints for nested entities.

Trade-off: GraphQL prevents over-fetching and minimizes payloads but introduces backend query parsing complexity. RESTful is simple and easily cached at CDN boundaries.

M3.3: WebSockets vs Server-Sent Events (实时推送抉择)

Theory: WebSockets establish full-duplex TCP tunnels. Server-Sent Events (SSE) offer unidirectional server-to-client streaming over HTTP.

Details: WebSockets upgrade connections via 'Upgrade: websocket'. SSE streams text using 'Content-Type: text/event-stream' with built-in client reconnect handlers.

Trade-off: WebSockets fit real-time collaborative applications. SSE is simpler, runs over standard HTTP, and is perfect for unidirectional streaming (e.g., AI chat logs).

M3.4: Protocol Buffers vs JSON vs Avro (数据编码抉择)

Theory: JSON is self-describing text. Protobuf uses tag-number binary indexing. Avro is a schema-coupled binary layout without mandatory code-generation.

Details: JSON parsing requires expensive runtime string tokenization. Protobuf encodes keys as integer tags using Varint formats. Avro embeds schemas or sends them sideband, appending raw values.

Trade-off: JSON is readable and universal. Protobuf is compact and typed. Avro is perfect for large analytical records but requires schema coordination.

M3.5: Dubbo vs gRPC (微服务 RPC 框架抉择)

Theory: Dubbo is a Java-centric framework providing service governance. gRPC is a multi-language transport contract framework.

Details: Dubbo integrates with ZooKeeper/Nacos for proxying, routing, and degradation. gRPC provides connection channels, leaving governance to service meshes like Istio.

Trade-off: Dubbo is plug-and-play for Java teams. gRPC is language-neutral but requires additional mesh components for microservice routing.

M4.2: etcd vs ZooKeeper (分布式协调引擎)

Theory: etcd is a Go/Raft KV store designed for cloud platforms. ZooKeeper is a Java/ZAB hierarchical node coordination system.

Details: etcd exposes gRPC APIs with MVCC history tracking and Watchers. ZooKeeper maintains an in-memory node tree (ZNodes) using persistent TCP sessions.

Trade-off: etcd is native to Kubernetes and easily containerized. ZooKeeper is robust for legacy Java clusters but JVM memory management is heavy.

M4.3: Strong Consistency vs Eventual (一致性折中)

Theory: Strong consistency forces all reads to return the latest write (Linearizability). Eventual consistency allows temporary divergences, converging when writes stop.

Details: Strong consistency relies on consensus logs or 2PC locks. Eventual consistency uses Quorum parameters ($R+W>N$) or CRDT models to resolve conflicts.

Trade-off: Strong consistency prevents stale reads but raises latency. Eventual consistency maximizes write availability but client runtimes must handle stale data.

M5.1: Vector Clocks vs Hybrid Logical Clocks (因果时钟)

Theory: Vector Clocks track causality using node version arrays. HLCs combine physical timestamps with logical counters into compact scalars.

Details: Vector Clocks scale in size with the number of nodes. HLCs fit inside a 64-bit or 96-bit integer, tracking physical-time drift.

Trade-off: Vector Clocks capture all concurrent state branches but payload sizes swell. HLCs are compact but cannot distinguish true concurrent updates within the same tick.

M3.6: Kong vs Envoy (API 网关选型)

Theory: Kong is an API gateway built on Nginx and OpenResty Lua scripts. Envoy is a C++ proxy designed for cloud-native sidecars and edge routing.

Details: Kong routes configs using relational DBs or static YAML. Envoy processes configuration changes dynamically via xDS control plane protocols without service restarts.

Trade-off: Kong is mature for traditional access limits and OAuth plugins. Envoy is built for service mesh dynamic meshes but configuration is complex.

M4.4: Gossip Protocol vs Centralized (服务发现通信)

Theory: Gossip protocols spread node state randomly to achieve peer convergence. Centralized registries expect clients to register and query heartbeats.

Details: Gossip uses UDP/TCP (e.g., in Consul member lists) scaling logarithmically. Registries (e.g. Nacos) require periodic client heartbeats and pull full tables.

Trade-off: Gossip has no single point of failure but state convergence takes time. Registries are accurate but broker network splits paralyze service discovery.

M4.1: Raft vs Paxos (强一致共识抉择)

Theory: Raft synchronizes logs using a strong Leader model. Paxos is a symmetric peer-to-peer consensus protocol using two-phase consensus proposals.

Details: Raft splits consensus into Leader Election, Log Replication, and Safety. Paxos permits out-of-order log writes, resolving conflicts via commit phases.

Trade-off: Raft is easy to understand and build. Paxos is mathematically robust but notoriously difficult to implement in production without bugs.

M4.5: Redlock vs ZK Ephemeral Node (分布式锁抉择)

Theory: Redlock coordinates locking across independent Redis hosts using timeouts. ZooKeeper locks rely on ephemeral node uniqueness and FIFO watch queues.

Details: Redlock executes 'SET NX' on multiple Redis instances. ZooKeeper creates sequential ephemeral nodes, blocking threads until the previous node is deleted.

Trade-off: Redlock is fast but clock drift or GC pauses risk double-locking. ZooKeeper locks are highly safe but node creation incurs consensus overhead.

M5.2: Sync IPC vs Async Event-Driven (服务协作选型)

Theory: Sync IPC blocks the client thread until the server returns. Async Event-Driven Architecture (EDA) decouples steps using message brokers.

Details: Sync calls propagate timeouts and can trigger cascade failures. Async MQ models buffer write surges, handling deliveries via consumer groups.

Trade-off: Sync calls are easy to trace. Async EDA is highly resilient but managing transactions and debugging distributed states is hard.

M5.3: Sidecar Service Mesh vs Proxyless (网格治理选型)	M5.5: Blue-Green vs Canary vs Rolling (发布策略抉择)
Theory: Sidecars intercept application network traffic using Envoy proxies. Proxyless gRPC implements xDS clients directly inside the SDK.	Theory: Blue-green maintains two identical versions, switching traffic via load balancers. Canary routes a small slice of traffic to test the new code. Rolling replaces pods incrementally.
Details: Sidecars use iptables or eBPF, transparent to application code. Proxyless gRPC queries xDS control planes (like Istio) directly.	Details: Blue-green shifts traffic instantly. Canary routes by ratio (e.g. 5%) or cookie. Rolling controls containers using 'maxSurge' and 'maxUnavailable' parameters.
Trade-off: Sidecar is language-agnostic but adds latency and memory overhead. Proxyless is zero-latency but locks applications to supported gRPC languages.	Trade-off: Blue-green is safest but requires double the infrastructure resource costs. Canary limits blast radius but requires backward-compatible database schemas. Rolling is resource-efficient but slow.

Zone T1: Opinionated CLI & Specs	
MySQL MVCC stack, PostgreSQL heap index, Consistent Hash	
Ring SD, Kafka PageCache, RabbitMQ backpressure, gRPC	
multiplexing, HLC Logical Clock	
Zone T2: Core Trade-off Matrix	
MySQL_Postgres_MVCC_locking,	Mon-
goDB_Cassandra_NoSQL_consensus,	Re-
dis_Memcached_caching_Slab,	In-
fluxDB_TimescaleDB_TSDB_cardinality,	
RocksDB_LSM_Column_Families,	
CockroachDB_TiDB_TSO_NewSQL,	
Kafka_Pulsar_BookKeeper_decoupled,	
Raft_Paxos_consensus_split	

M5.4: Serverless (FaaS) vs Containerized (K8s) (计算部署选型)
Theory: FaaS scales resources on-demand per request event. Containerized Kubernetes manages long-lived pods running continuous loops.
Details: FaaS runs short-lived functions, billing by execution time. K8s scales pods using HPAs based on resource averages.
Trade-off: FaaS is cost-effective for low-utilization tasks but suffers from cold-start latency. K8s fits high-traffic systems but has fixed cost overhead.